

Lab 7: Database Connectivity — Game Catalog CRUD

ส่วนที่ 1: Software Design & Principles Explanation

GRASP Patterns

Information Expert — มอบหมายหน้าที่ให้คลาสที่ "รู้ข้อมูล" มากที่สุด **Game** entity เก็บ field ทั้งหมด (price, discountType) จึงเป็นคนที่ควรเก็บข้อมูลของตัวเอง แต่การ "คำนวณ" ราคาส่วนลดไม่ได้อยู่ใน **Game** — เพราะ **Game** ไม่รู้วิธีคำนวณส่วนลดแต่ละแบบ ผู้เชี่ยวชาญด้านนี้คือ **DiscountStrategy** แต่ละตัว ซึ่งรู้สูตรคำนวณของตัวเองโดยเฉพาะ (10%, 20%, 0%)

Controller Pattern — **GameController** ทำหน้าที่เป็นตัวกลางรับ HTTP request จาก UI แล้วส่งต่อไปให้ระบบจัดการ โดยตัว Controller เองไม่ทำ business logic ใดๆ เป็นไปตามหลักการที่ว่า Controller ควรเป็นแค่ "ทางผ่าน" ไม่ใช่ผู้ลงมือทำงานจริง

High Cohesion — แต่ละคลาสมีหน้าที่แคบและสัมพันธ์กันแน่นภายในตัวเอง เช่น **StudentDiscountStrategy** มีหน้าที่เดียวคือคำนวณส่วนลด 10% ไม่ปะปนกับการดึงข้อมูลหรือรับ HTTP request เลย ทำให้อ่านง่าย แก้ไขจุดเดียวจบ

Low Coupling — แต่ละ Layer ผูกกันผ่าน interface/abstraction เท่านั้น ไม่ผูกกับ implementation ตรงๆ เช่น **GameService** ไม่รู้จัก **StudentDiscountStrategy** โดยตรง รู้จักแค่ **DiscountContext** และ **DiscountStrategy** (interface) เท่านั้น ถ้าเปลี่ยนวิธีคำนวณส่วนลดภายใน ก็ไม่กระทบ Service เลย

Indirection — **DiscountContext** ทำหน้าที่เป็นตัวกลาง (mediator) ระหว่าง **GameService** กับ Concrete Strategies ทั้ง 3 ตัว ทำให้ Service ไม่ต้องผูกติดกับ Strategy ตัวใดตัวหนึ่งโดยตรง เพิ่มชั้นกันกระแทก (buffer) ระหว่างสองส่วนที่ไม่ควรรู้จักกันตรงๆ

SOLID Principles

S — Single Responsibility Principle

ทุกคลาสมีเหตุผลในการเปลี่ยนแปลงเพียงเหตุผลเดียว: **GameRepository** เปลี่ยนเมื่อวิธี

เข้าถึงฐานข้อมูลเปลี่ยน, **GameService** เปลี่ยนเมื่อ business rule เปลี่ยน, **GameController** เปลี่ยนเมื่อ routing/HTTP handling เปลี่ยน, **DiscountStrategy** แต่ละตัวเปลี่ยนเมื่อสูตรส่วนลดนั้นๆ เปลี่ยน

O — Open/Closed Principle

ระบบ เปิดให้ขยาย แต่ปิดการแก้ไข ชัดเจนที่สุดใน Strategy Pattern — ถ้าต้องการเพิ่มส่วนลด VIP 30% เพียงสร้างคลาสใหม่ **VipDiscountStrategy** implements **DiscountStrategy** แล้วเพิ่ม case ใน **DiscountContext.resolveStrategy()** โดยไม่ต้องแตะโค้ด **GameService**, **GameController**, หรือ Strategy เดิมที่ทำงานอยู่แล้วเลย

L — Liskov Substitution Principle

NoDiscountStrategy, **StudentDiscountStrategy**, **SeasonalSaleStrategy** ทุกตัวสามารถใช้แทนกันได้อย่างสมบูรณ์ผ่านชนิด **DiscountStrategy** — **DiscountContext** เรียก **calculatePrice()** โดยไม่สนใจว่าเบื้องหลังเป็น Strategy ตัวไหน ทุกตัวรับประกันพฤติกรรมตรงตาม contract (คืนค่า **BigDecimal** ที่ไม่ null เสมอ)

I — Interface Segregation Principle

DiscountStrategy มีเพียง 2 เมธอด (**calculatePrice**, **getName**) ที่จำเป็นจริงๆ ไม่ยัดเมธอดที่ไม่เกี่ยวข้อง (เช่น การบันทึกฐานข้อมูล) เข้าไปใน interface นี้ ทำให้ผู้ implement ไม่ต้องเขียนเมธอดที่ไม่ได้ใช้

D — Dependency Inversion Principle

Layer บนไม่พึ่งพา Layer ล่างโดยตรง แต่พึ่งพา abstraction ทั้งคู่: **GameController** พึ่งพา **GameService** (ไม่รู้จัก Repository), **GameService** พึ่งพา **GameRepository** (interface) และ **DiscountContext** ผ่าน Constructor Injection — Spring เป็นคนฉีด (inject) implementation จริงเข้ามาให้ตอน runtime ทำให้สลับ implementation ได้โดยไม่กระทบโค้ดที่เรียกใช้

Strategy Pattern ในการคำนวณส่วนลด

โครงสร้างประกอบด้วย 3 ส่วนตามแบบแผน Strategy Pattern มาตรฐาน:

1. **DiscountStrategy** (Strategy interface) — สัญญากลางที่บังคับให้ทุกกลยุทธ์ต้องมี **calculatePrice()** และ **getName()**
2. Concrete Strategies — **NoDiscountStrategy** (0%), **StudentDiscountStrategy** (10%), **SeasonalSaleStrategy** (20%) แต่ละตัว

implement สูตรคำนวณของตัวเองแยกจากกันสมบูรณ์ ไม่มีการเขียน if-else สูตรทั้งหมดไว้ในที่เดียว

3. **DiscountContext** (Context) — ตัวกลางที่รับ **discountType** (string จากฟอร์ม) แล้วเลือก (**resolveStrategy**) ว่าจะมอบหมายงานคำนวณให้ Strategy ตัวไหน จากนั้น "ส่งต่อ" (delegate) การทำงานจริงให้ Strategy นั้นรับผิดชอบ

ประโยชน์ด้าน OCP ที่ชัดเจนที่สุด: ถ้าไม่ใช้ Strategy Pattern ตรรกะการคำนวณส่วนลดทั้งหมดจะต้องกระจุกอยู่ใน **if/else** หรือ **switch** ก่อนใหญ่ก่อนเดียวใน Service — ทุกครั้งที่เพิ่มส่วนลดใหม่ต้องแก้โค้ดเดิมที่ทำงานอยู่แล้ว เสี่ยงทำของเดิมพัง (regression) แต่ด้วย Strategy Pattern การเพิ่มส่วนลดใหม่คือการ "เดิมของใหม่" (สร้างคลาสใหม่) ไม่ใช่ "แก้ของเก่า" — ตรงตามนิยาม OCP: "เปิดรับการขยาย ปิดรับการแก้ไข" อย่างแท้จริง

ทำไมต้องแยก Service Layer ออกจาก Controller และ Repository

ถ้าไม่มี Service Layer — Controller จะต้องทำทั้งสองอย่างพร้อมกัน: รับ HTTP request และ ดัดสินใจเรื่อง business logic (เช่น คำนวณส่วนลด) ทำให้ Controller คลาสเดียวมีเหตุผลในการเปลี่ยนแปลงหลายเหตุผล (ผิด SRP) และ coupling สูง เพราะ Controller ต้องรู้จักทั้ง **GameRepository** และ **DiscountContext** โดยตรง

การมี Service Layer คั่นกลาง ให้ประโยชน์ 2 ด้าน:

- Low Coupling — Controller รู้จักแค่ **GameService** ตัวเดียว ไม่ต้องรู้ว่าเบื้องหลังมี Repository หรือ Strategy Pattern ทำงานอยู่กี่ชั้น หาก business logic เปลี่ยน (เช่น เพิ่มการ validate ราคาห้ามติดลบ) แก้แค่ใน Service โดย Controller ไม่ต้องเปลี่ยนแม้แต่บรรทัดเดียว
- High Cohesion — งานที่เกี่ยวข้องกัน (ดึงข้อมูล + คำนวณส่วนลด + จัดการ transaction) ถูกรวมไว้ในที่เดียวคือ **GameService** ทำให้ business rule ทั้งหมดของระบบอยู่จุดเดียว ง่ายต่อการหาและแก้ไข ไม่กระจัดกระจายไปตาม Controller หลายๆ ตัว

พูดง่ายๆ คือ Controller จัดการ "HTTP", Repository จัดการ "ฐานข้อมูล", ส่วน Service คือที่ที่ "กฎทางธุรกิจ" อยู่ — แยกกันชัดเจนตาม concern ของแต่ละคลาส

Execution Flow (ตัวอย่าง: เพิ่มเกมใหม่พร้อมส่วนลดนักศึกษา)

1. Browser to Controller: ผู้ใช้กรอกฟอร์ม `add.html` แล้วกด "บันทึก" แล้ว ส่ง `POST /games/save` พร้อมข้อมูลฟอร์มไปยัง `GameController.saveGame()`
2. Controller to Service: Spring แปลง form data เป็น object `Game` (ผ่าน `@ModelAttribute`) แล้ว Controller เรียก `gameService.saveGame(game)` — ส่งต่อทันทีโดยไม่แตะ business logic
3. Service to Repository: `GameService.saveGame()` เรียก `gameRepository.save(game)` Spring Data JPA แปลง object เป็นคำสั่ง SQL `INSERT` และส่งไปบันทึกจริงที่ PostgreSQL ผ่าน Hibernate
4. กลับมาที่ Controller to Redirect: หลังบันทึกสำเร็จ Controller ส่ง `redirect:/games` กลับไปหน้ารายการเกม
5. Browser to Controller (รอบใหม่): เบราวเซอร์ยิง `GET /games` เข้ามาที่ `GameController.listGames()`
6. Controller to Service, Service to Repository: Controller เรียก `gameService.getAllGames()` Service เรียก `gameRepository.findAll()` แล้วดึงข้อมูลเกมทั้งหมดจาก PostgreSQL กลับมาเป็น `List<Game>`
7. Service to Strategy Pattern: ก่อนส่งกลับ Service วนลูปเรียก `applyDiscount()` กับทุกเกม โดยเรียก `discountContext.calculateFinalPrice(discountType, price)` `DiscountContext` เช็คค่า `discountType` ("STUDENT") แล้วเลือกใช้ `StudentDiscountStrategy` แล้วคำนวณ `price - 10%` แล้วคืนราคาสุทธิกลับมา
8. Service to Controller, Controller to View: Service เช็คค่า `finalPrice/discountName` ที่คำนวณได้ลงใน object `Game` (แบบ `@Transient` ไม่กระทบฐานข้อมูล) แล้วส่งกลับให้ Controller ใส่ลง `Model` แล้วเลือก view `games/list`
9. View to Browser: Thymeleaf render `list.html` โดยดึงค่า `game.finalPrice` มาแสดงเป็นราคาสุทธิในตาราง ส่งกลับเป็น HTML ให้เบราว์เซอร์แสดงผลสุดท้าย

ส่วนที่ 2: Code Implementation & Explanation

ภาพรวมสถาปัตยกรรม (Layered Architecture)

โปรเจกต์นี้ออกแบบตามสถาปัตยกรรมแบบเป็นชั้น (Layered Architecture) แบ่งความรับผิดชอบออกเป็น 4 ชั้นหลัก โดยแต่ละชั้นสื่อสารกับชั้นถัดไปผ่าน Constructor Injection เท่านั้น ทำให้แต่ละคลาสไม่ต้องรู้วิธีการสร้าง object ของ dependency ตัวเอง (Dependency Inversion Principle):

- Controller (Presentation Layer) — รับ HTTP request และเลือก View

- Service (Business Logic Layer) — ตรรกะทางธุรกิจ และเรียกใช้ Strategy Pattern
- Repository (Data Access Layer) — ติดต่อฐานข้อมูลผ่าน Spring Data JPA
- Strategy Package — กลยุทธ์การคำนวณส่วนลดที่สลับเปลี่ยนได้ (Strategy Pattern)

การไหลของข้อมูล: Controller to Service to Repository/DiscountContext to Database

1. Entity: Game.java

Entity คือคลาสที่แมปกับตาราง games ในฐานข้อมูล PostgreSQL โดยใช้ JPA Annotations:

- @Entity และ @Table(name = "games") — ระบุว่าคลาสนี้คือตารางในฐานข้อมูล
- @Id, @GeneratedValue(strategy = GenerationType.IDENTITY) — กำหนด primary key ที่เพิ่มค่าอัตโนมัติ
- @Column — แมป field กับคอลัมน์ในตาราง เช่น release_date, discount_type
- @Transient — ใช้กับ discountName และ finalPrice ซึ่งเป็นค่าที่คำนวณขึ้นชั่วคราวผ่าน Strategy Pattern ทุกครั้งที่ดึงข้อมูล จึงไม่ต้องการให้ Hibernate บันทึกลงฐานข้อมูลจริง

2. Repository: GameRepository.java

ชั้น Data Access Layer ใช้ Spring Data JPA เพียงแค่ extends JpaRepository<Game, Long> ก็จะได้เมธอด CRUD พื้นฐานทั้งหมด (findAll, findById, save, deleteById) มาโดยอัตโนมัติ โดยไม่ต้องเขียน SQL เอง

หลักการ SRP (Single Responsibility Principle): คลาสนี้รับผิดชอบเฉพาะการเข้าถึงฐานข้อมูลเท่านั้น ไม่มี business logic ปะปนอยู่

3. Strategy Package (Strategy Pattern)

หัวใจของ Lab นี้คือ Strategy Pattern ที่ใช้คำนวณราคาส่วนลด ประกอบด้วย 3 ส่วน:

3.1 DiscountStrategy.java (Interface)

Interface กลางที่กำหนดสัญญา (contract) ว่าทุก Concrete Strategy ต้องมีเมธอด calculatePrice() และ getName()

3.2 Concrete Strategies

มีการ implement 3 กลยุทธ์ ทำเครื่องหมาย @Component ให้ Spring จัดการเป็น Bean:

- NoDiscountStrategy — ราคาปกติ ไม่หักส่วนลด (0%)
- StudentDiscountStrategy — หักส่วนลด 10% ด้วย BigDecimal เพื่อความแม่นยำของตัวเลขทศนิยม
- SeasonalSaleStrategy — หักส่วนลด 20%

3.3 DiscountContext.java (Context)

Context ทำหน้าที่เลือก Strategy ที่เหมาะสมโดยอัตโนมัติจากค่า discountType (NONE/STUDENT/SEASONAL) แล้วมอบหมาย (delegate) การคำนวณให้ Strategy นั้น ๆ ทำแทน

ประโยชน์ของ Strategy Pattern ในจุดนี้ (Open/Closed Principle): หากต้องการเพิ่มส่วนลดใหม่ เช่น VipDiscountStrategy (30%) เพียงสร้างคลาสใหม่ที่ implement DiscountStrategy และเพิ่ม case ใน resolveStrategy โดย ไม่ต้องแก้ไขโค้ดเดิมที่ทำงานอยู่แล้ว

4. Service: GameService.java

ชั้น Business Logic เชื่อมระหว่าง Repository และ Strategy Pattern เข้าด้วยกัน โดยรับทั้งสองอย่างผ่าน Constructor Injection

Service ไม่รู้เลยว่า DiscountContext เลือก Strategy ตัวไหนภายใน เพียงเรียกใช้ผ่าน interface เท่านั้น นี่คือการ Dependency Inversion Principle (DIP)

5. Controller: GameController.java

ชั้นบนสุดที่รับ HTTP request จากผู้ใช้ แมป URL ไปยังเมธอดต่าง ๆ (RESTful-style routing) และส่งต่อให้ GameService จัดการ โดยรับ GameService ผ่าน Constructor Injection

Controller ไม่มี business logic หรือคำสั่งเข้าถึงฐานข้อมูลใด ๆ เลย มีหน้าที่แค่รับ input ส่งต่อ เลือก view เท่านั้น (SRP)

ส่วนที่ 3: Web Application & Database Screenshots

3.1 หน้าจอการเพิ่มเกมใหม่ (Create)

The screenshot shows the 'Add New Game' form in the Game Catalog application. The form is titled 'เพิ่มเกมใหม่' (Add New Game) and includes a back link '← กลับไปรายการเกม'. Below the title is a note: 'กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ' (Enter game information to add to the system). The form fields are:

- ชื่อเกม (Game Name):** Maid of Sker-673380275-5-section2
- แนวเกม (Genre):** Horror
- แพลตฟอร์ม (Platform):** PC
- คะแนน (Rating):** 9.3
- ราคาปกติ (บาท) (Regular Price (Baht)):** 319
- % ส่วนลด (Strategy) (% Discount (Strategy)):** ราคาปกติ (0%)
- วันวางจำหน่าย (Release Date):** 07 / 28 / 2020

At the bottom of the form are two buttons: 'บันทึก' (Save) and 'ยกเลิก' (Cancel).

รูปที่ 3.1: หน้าฟอร์มเพิ่มเกมใหม่

3.2 หน้าจอแสดงรายการเกมทั้งหมด (Read)

The screenshot shows the 'All Games' list in the Game Catalog application. The page title is 'รายการเกมทั้งหมด' (All Games) with a subtitle 'จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete'. A green success message at the top states: 'เพิ่มเกม "Maid of Sker-673380275-5-section2" สำเร็จ' (Add game "Maid of Sker-673380275-5-section2" successful). A 'เพิ่มเกมใหม่' (Add New Game) button is in the top right. The table below lists the games:

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	NieR Replicant	Action, RPG	PC	★ 8.7	23/04/2023	ส่วนลดนักศึกษา 10%	1791.00 1999.00	 
2	Maid of Sker-673380275-5-section2	Horror	PC	★ 9.3	28/07/2020	ราคาปกติ	319.00	 

At the bottom of the table, it says 'ทั้งหมด 2 เกม' (Total 2 games). The footer of the page reads: 'CP353002 Principles of Software Design — Lab 7: Database Connectivity'.

รูปที่ 3.2: หน้าแสดงรายการเกมทั้งหมด พร้อมแสดงราคาสุทธิที่คำนวณผ่าน Strategy Pattern

3.3 หน้าจอแก้ไขข้อมูลเกม (Update)

http://localhost:8080/games

🔍 ⭐

VPN

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

📁 รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

⊕ เพิ่มเกมใหม่

✅ อัปเดตเกม "Maid of Sker-673380275-5-section2" สำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	NieR Replicant	Action, RPG	PC	★ 8.7	23/04/2023	ส่วนลดนักศึกษา 10%	1791.00 1999.00	✎ 🗑
2	Maid of Sker-673380275-5-section2	Horror	PC	★ 9.3	28/07/2020	ส่วนลดนักศึกษา 10%	287.10 319.00	✎ 🗑

📄 ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

รูปที่ 3.3: หน้าฟอร์มแก้ไขข้อมูลเกม

3.4 หน้าจอยืนยันและการลบข้อมูลเกม (Delete)

← กลับไปรายการเกม

 ยืนยันการลบ



คุณแน่ใจหรือไม่?

การดำเนินการนี้จะลบข้อมูลเกมออกจากระบบอย่างถาวร

ชื่อเกม

ร่างวันลอยกระทง

แนวเกม

Dance

แพลตฟอร์ม

Onsite, บังคับซื้อ

คะแนน

★ 5.0

ราคา

300.00 บาท

 ยืนยันลบ

 ยกเลิก

http://localhost:8080/games

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

 รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

 เพิ่มเกมใหม่

 ลบเกมสำเร็จ

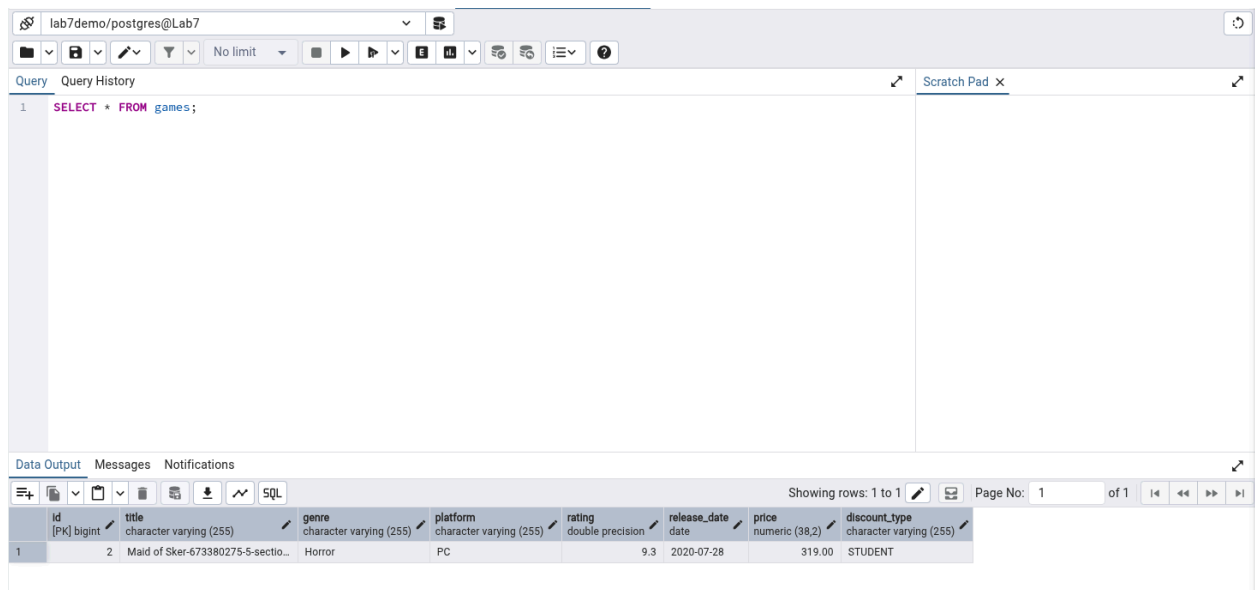
#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Maid of Sker-673380275-5-section2	Horror	PC	★ 9.3	28/07/2020	ส่วนลดนักศึกษา 10%	287.10 319.00	 

📄 ทั้งหมด 1 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

รูปที่ 3.4: หน้าจอยืนยันการลบเกม และผลลัพธ์หลังทำการลบ

3.5 หน้าจอตรวจสอบข้อมูลใน PostgreSQL Database



The screenshot displays a PostgreSQL database management interface. At the top, the connection is identified as 'lab7demo/postgres@Lab7'. Below this, a toolbar contains various icons for file operations, query execution, and settings. The main area is divided into two panes: 'Query' and 'Query History'. The 'Query' pane shows the SQL statement: `SELECT * FROM games;`. The 'Query History' pane is empty. To the right of the query panes is a 'Scratch Pad' tab. Below the query panes, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with 1 row and 8 columns. The table headers are: `id` (PK, bigint), `title` (character varying (255)), `genre` (character varying (255)), `platform` (character varying (255)), `rating` (double precision), `release_date` (date), `price` (numeric (38,2)), and `discount_type` (character varying (255)). The single row of data shows: 1, Maid of Sker-673380275-5-sectio..., Horror, PC, 9.3, 2020-07-28, 319.00, and STUDENT.

id	title	genre	platform	rating	release_date	price	discount_type
1	Maid of Sker-673380275-5-sectio...	Horror	PC	9.3	2020-07-28	319.00	STUDENT

รูปที่ 3.5: ข้อมูลตาราง games ที่ถูกจัดเก็บจริงในฐานข้อมูล PostgreSQL